

Verifieringsrapport — CVmatchen

Datum: 2026-07-07 Utförd av: automatiserad granskning (verktygsbaserad) på uppdrag av PathfinderAI. Ej ersättning för oberoende pentest/WCAG-audit — se avsnitt "Vad detta INTE är".

Syftet är att ersätta obekräftade påståenden med **mätdata**, som svar på den återkoppling som lyfte att tidigare kvalitetsintyg var självskattade.

Sammanfattning

Område	Metod	Resultat
Hemligheter i kod	Mönstersökning (nyckelformat)	✅ Inga hårdkodade nycklar — allt via miljövariabler
Beroende-sårbarheter	<code>npm audit</code>	⚠ 23 kända sårbarheter (4 höga) i PDF-subsystemet (puppeteer/ws)
Tillgänglighet (statisk)	WCAG-heuristik	✅ Grundstruktur god · ⚠ enstaka fält/knappar för manuell kontroll

1. Hemligheter i kod ✅

Metod: sökning efter kända nyckelmönster (Anthropic `sk-ant-` , AWS `AKIA...` , Supabase `service_role` /JWT `eyJ...`) i all JS/HTML/JSON.

Resultat: inga hårdkodade nycklar. Samtliga hemligheter läses via `process.env` och lagras krypterat i Vercel. Detta bekräftar att en läcka av källkoden inte i sig exponerar produktionsnycklar.

2. Beroende-sårbarheter ⚠

Metod: `npm audit` mot produktionsberoenden.

Resultat: 23 kända sårbarheter (19 medel, 4 höga). **Samtliga** ligger i en enda beroendekedja:

```
puppeteer-core → @puppeteer/browsers → ws / tar-fs
```

Denna kedja används av `api/pdf.js` för PDF-generering (headless Chromium). De höga CVE:erna gäller `ws` (WebSocket-bibliotek): DoS vid många HTTP-headers, minnesexponering, minnesuttömning.

Riskbedömning: subsystemet körs serverside vid PDF-export och tar inte emot fientlig extern WebSocket-trafik i normal drift → praktisk exponering är begränsad. Men känd-sårbara paket ska ändå åtgärdas.

Åtgärd: uppdatera `puppeteer-core` till 25.x (brytande ändring → kräver test av PDF-exporten). Planeras i backlogen. Övriga beroenden (`@sentry/node` , `@supabase/supabase-js`) är rena.

3. Tillgänglighet — statisk granskning

Metod: heuristisk WCAG 2.1-kontroll av samtliga vyer: språkattribut, viewport, rubrikstruktur, alt-texter, formuläretiketter, knapp-text.

Vy	lang	viewport	<h1>	Bilder utan alt	Fält utan tydlig etikett	Tomma knappar
index.html (deltagare mobil)	✓	✓	1 ✓	0 ✓	2 / 177	2
handlaggare.html (admin)	✓	✓	2 ✓	0 ✓	6 / 23	12
desktop.html (deltagare)	✓	✓	1 ✓	0 ✓	1 / 31	0
tillganglighet.html	✓	✓	1 ✓	0 ✓	0 ✓	0 ✓

Tolkning: - Grunden är god: språkattribut, viewport, rubriknivåer och alt-texter är på plats i alla vyer. - **Åtgärdat 2026-07-07:** samtliga flaggade interaktiva kontroller (ikon-knappar för favorit/foto, sök- och inbjudningsfält, filuppladdning) har försetts med `aria-label`. Efter fix: **0 synliga kontroller utan textalternativ**. Kvar är en enda `display:none`-legacyknapp som aldrig når skärmläsare (borttags i städning, backlog #36). - **Kvarstår för extern audit:** uppmätt färgkontrast, skärmläsartest (NVDA/VoiceOver) och tangentbordsgenomgång — kräver riktig hjälpmedelsmiljö, se nedan.

Vad detta INTE är

Denna rapport är **automatiserad och verktygsbaserad**. Den ersätter inte:

- **Oberoende penetrationstest** — aktiv angreppssimulering av extern part.
- **Fullständig WCAG-audit** — skärmläsartest (NVDA/VoiceOver), tangentbordsgenomgång, uppmätt färgkontrast.
- **Lasttest** — verifiering av kapacitetspåståenden under verklig last.

Dessa kräver antingen extern part eller en pilotmiljö och är planerade (se teknisk specifikation, avsnitt 12, samt DPIA:ns åtgärdsplan). Denna rapport visar dock att de påståenden som *går* att verifiera automatiskt håller, och ger en ärlig bild av var kvarvarande arbete finns.

Åtgärdslista från denna granskning

#	Åtgärd	Status
1	<code>aria-label</code> på flaggade ikon-knappar och fält	✓ Klart 2026-07-07
2	Lasttest-script framtaget (<code>tests/load-test.js</code> , k6)	✓ Klart — kör mot preview
3	Uppgradera puppeteer-core + <code>@sparticuz/chromium</code> (23 dep-sårbarheter) + testa PDF	⌚ Backlog #35 (kräver PDF-test före prod)
4	Boka oberoende pentest (finansieras via Visionsfonden)	⌚ Hög — före skarp drift
5	Boka extern WCAG-audit (t.ex. Funka)	⌚ Hög — lagkrav DOS-lagen

Lasttest — så bevisas skalningen

Ett körklart k6-script finns i `tests/load-test.js`. Det trappar upp till 500 samtidiga virtuella användare mot publika sidor (ingen persondata) och mäter svarstid + felandel. Grön tröskel (p95 < 1,5 s, fel < 1 %) ger konkret bevis för kapacitetspåståendet. Körs mot en preview-miljö, ej skarp prod.